IMPLEMENTAÇÃO DE INTERNAL DEVELOPER PLATFORM PARA GOVERNANÇA EM AMBIENTE ENTERPRISE

Eduardo Doval Zenardi Filho

SUMÁRIO EXECUTIVO

O presente projeto propõe a arquitetura e implementação de um *Internal Developer Platform* (IDP) para uma organização multinacional de grande porte. Do ponto de vista dos usuários – compostos por desenvolvedores de software, engenheiros de dados e profissionais de segurança corporativa – o sistema atua como um portal centralizado de *self-service* para o provisionamento automatizado de infraestrutura e serviços em nuvem.

O sistema funcionará em um ambiente de nuvem distribuído globalmente (Microsoft Azure), servindo como a camada de abstração principal entre o código-fonte e a infraestrutura subjacente, como clusters Kubernetes (AKS) e bancos de dados. A principal função da plataforma é reduzir a carga cognitiva dos times de engenharia, oferecendo *templates* de arquitetura padronizados (*Golden Paths*) que já contêm as melhores práticas de Infraestrutura como Código (IaC), CI/CD, monitoramento e governança de segurança embutidas.

PROBLEMATIZAÇÃO

Em ambientes corporativos de escala global, a proliferação de microsserviços e a adoção de múltiplas soluções em nuvem geram uma complexidade operacional insustentável. Atualmente, os times de desenvolvimento gastam uma parcela significativa de seu tempo configurando infraestrutura, gerenciando permissões de acesso e lidando com armazenamento de dados distribuídos, em vez de focar na entrega de valor de negócio.

Além da ineficiência e do gargalo operacional, essa falta de padronização arquitetural resulta em graves riscos de segurança. Sem uma governança centralizada, aplicações são

frequentemente implantadas com configurações de rede inadequadas, falta de rastreabilidade de logs e violações do princípio de privilégio mínimo. A ausência de um fluxo automatizado e validado dificulta a auditoria, a aplicação de *patches* de segurança em escala e a manutenção de uma postura de nuvem resiliente contra ameaças cibernéticas.

JUSTIFICATIVA E OBJETIVO GERAL

Justificativa

Um *Internal Developer Platform* (IDP) é, fundamentalmente, uma camada de autoatendimento (*self-service*) construída para abstrair a complexidade operacional da infraestrutura de nuvem. Para uma organização *enterprise*, sua relevância é inestimável: ela atua como o motor central que traduz as intenções dos desenvolvedores em infraestrutura provisionada, permitindo que as equipes foquem na escrita de código de negócio em vez de gerenciar configurações de servidores, redes e permissões.

A implementação dessa plataforma justifica-se pela necessidade crítica de otimizar a entrega de software e garantir a padronização em um cenário corporativo globalizado e de alta complexidade . Atualmente, a fragmentação de ferramentas e a configuração manual geram uma alta carga cognitiva para as equipes, resultando em atrasos no *time-to-market* de novos produtos e na elevação dos custos operacionais.

Para o negócio, o impacto desta intervenção é estratégico e transformador. O IDP centraliza a governança corporativa, assegurando que todas as aplicações sigam padrões rigorosos de arquitetura e segurança desde o início do desenvolvimento (*security by design*). Além de automatizar o provisionamento em nuvem, a plataforma estabelece diretrizes claras para os ambientes de trabalho — padronizando, inclusive, o uso de assistentes de codificação baseados em inteligência artificial. Dessa forma, o IDP mitiga falhas humanas, reduz vulnerabilidades sistêmicas e permite que as equipes foquem exclusivamente na inovação, impulsionando a competitividade da organização.

Objetivo Geral

Propor a arquitetura e o modelo de implementação de um *Internal Developer Platform* (IDP) escalável e segura para o contexto de uma corporação multinacional, visando automatizar o provisionamento de infraestrutura *on demand*, centralizar a governança e reduzir a carga cognitiva dos times, garantindo segurança na entrega contínua de software.

FUNDAMENTAÇÃO TEÓRICA

A construção de um *Internal Developer Platform* (IDP) de nível *enterprise* requer a orquestração de tecnologias nativas de nuvem e ferramentas consolidadas para garantir escalabilidade, segurança e resiliência. A arquitetura proposta, focada no ecossistema Microsoft Azure e em padrões do *Cloud Native Computing Foundation* (CNCF), fundamenta-se nos seguintes pilares:

Portal do Desenvolvedor, Catálogo de Serviços e *Golden Paths*

O Backstage atua como a interface central da plataforma. Ele unifica o catálogo de software corporativo e fornece os *Golden Paths*. Um *Golden Path* (caminho pavimentado) é um conjunto de práticas, *templates* e ferramentas pré-configuradas e suportadas oficialmente pela organização para o desenvolvimento de software. Sua importância reside em reduzir a fadiga de decisão técnica. Para estabelecer esses caminhos, a engenharia de plataforma deve tratar o IDP como um produto (*Platform as a Product*), colaborando continuamente com os desenvolvedores para criar *templates* que resolvam dores reais (como a criação rápida de uma API RESTful segura). Através de seus *Software Templates*, a plataforma abstrai a complexidade inicial de criação de microsserviços.

Nuvem Enterprise e Orquestração de Infraestrutura

O Microsoft Azure servirá como provedor base, com o AKS (*Azure Kubernetes Service*) como motor de orquestração de contêineres. O Crossplane é adotado como o motor de Infraestrutura como Código (IaC). Ao invés de fluxos imperativos externos, o Crossplane estende a API do Kubernetes, permitindo provisionar recursos complexos do Azure nativamente.

Automação, GitOps e Governança de IA

O Azure Pipelines atua como motor de CI, compilando o código, rodando testes automatizados, escaneando vulnerabilidades e publicando as imagens no *Azure Container Registry* (ACR). O ArgoCD atua como motor de CD (GitOps), garantindo que o estado da infraestrutura no AKS seja idêntico ao declarado nos repositórios. A automação também injeta nativamente arquivos de instrução (como o padrão AGENT.md do GitHub Copilot) na raiz de cada projeto, governando o uso de IA local pelos engenheiros.

Ciclo DevSecOps e Segurança (Cybersecurity)

A adoção do IDP e dos *Golden Paths* é o alicerce para materializar o ciclo DevSecOps. Ao invés de a segurança ser um gargalo no final do desenvolvimento, ela é injetada na origem (*Shift-Left*). Quando um time inicia um projeto via IDP, o pipeline gerado já herda validações rigorosas e políticas de conformidade automáticas. O uso obrigatório de imagens de contêiner *distroless* da Chainguard é um exemplo prático dessa governança: os projetos já nascem livres de vulnerabilidades conhecidas (CVEs) e sem *shells*, garantindo que a postura de cibersegurança da empresa escale na mesma velocidade do desenvolvimento de software. Complementarmente, o Kyverno atua como motor de *Policy as Code*, bloqueando implantações fora do padrão, e o *Azure Key Vault* gerencia segredos de forma segura via *CSI Driver*.

PERCURSO METODOLÓGICO DA INTERVENÇÃO

Arquitetura Lógica da Solução

Para tornar a IDP confiável e resiliente a mudanças, a arquitetura foi desenhada com forte separação de responsabilidades. Isso significa que o sistema é modular o suficiente para que cada pedaço de código tenha um escopo único, garantindo alta coesão e baixo acoplamento . Essa extensibilidade permite que a organização troque módulos, altere camadas e adicione novos serviços sem quebrar os contratos de dados subjacentes das outras ferramentas, mantendo a plataforma em execução mesmo durante atualizações críticas.

A plataforma será dividida em quatro camadas principais:

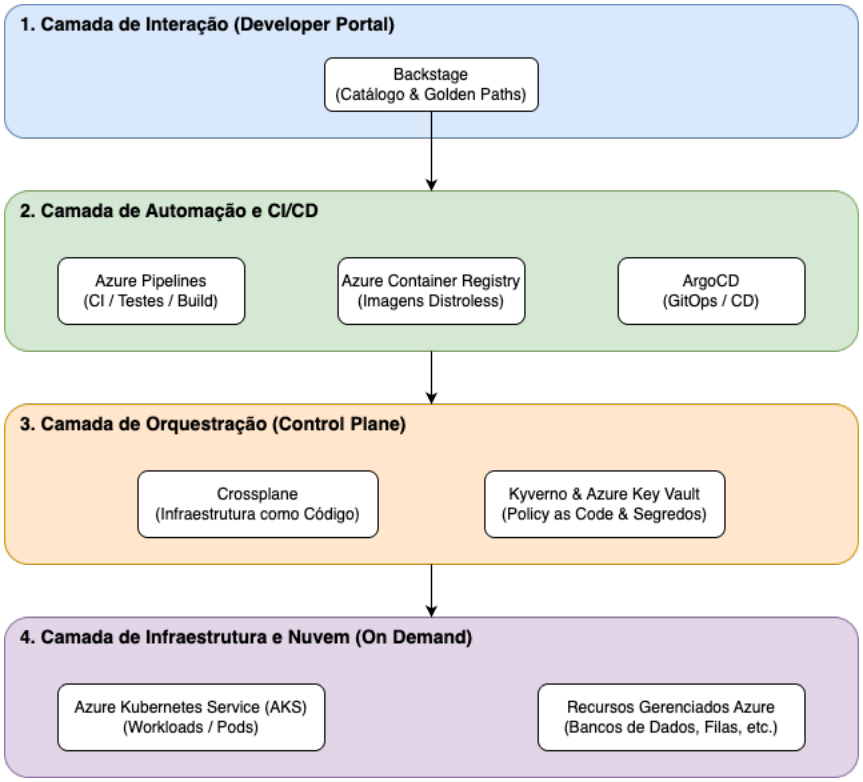

Figura 1: Modelo de Arquitetura Base

1. **Camada de Interação (*Developer Portal*):** Uma interface unificada onde os desenvolvedores podem descobrir serviços, consultar documentações e solicitar o provisionamento de novos recursos por meio de *Golden Paths*.

2. **Camada de Automação e CI/CD:** Responsável pela execução (pipelines no Azure Pipelines e sincronização via ArgoCD).
3. **Camada de Orquestração (*Control Plane*):** A decisão de hospedar o própria IDP de forma nativa no Kubernetes (AKS) garante uma escalabilidade ímpar para a plataforma. O Kubernetes permite que a arquitetura seja dimensionada horizontalmente, adicionando automaticamente novos *pods* e réplicas dos serviços do IDP nos horários de pico. Recebe as requisições via uma estrutura RESTful e traduz em comandos de automação baseados em manifestos processados pelo Crossplane.
4. **Camada de Infraestrutura e Nuvem (*On Demand*):** Onde os recursos reais são provisionados dinamicamente via código, permitindo escalabilidade.

Análise e Mitigação de Riscos

- **Risco 1: O IDP se tornar um Ponto Único de Falha (SPOF).** *Mitigação:* Desenhar o *Control Plane* em um ambiente de alta disponibilidade, utilizando múltiplos nós e zonas de disponibilidade na nuvem Azure.
- **Risco 2: Rejeição dos desenvolvedores (baixa adoção).** *Mitigação:* Tratar o IDP como um produto (*Platform as a Product*), envolvendo os usuários finais no desenho dos *templates* e garantindo que a plataforma reduza o tempo de *setup* local em vez de adicionar burocracia.

RESULTADOS ESPERADOS

A implementação do Internal Developer Platform (IDP) deverá promover uma transformação na cultura de engenharia, na postura de segurança e na eficiência financeira da organização . Espera-se alcançar os seguintes resultados:

- **Métricas de Aceleração de Entrega:** Com o provisionamento automatizado de infraestrutura e a adoção dos *Golden Paths*, estima-se um ganho operacional sistêmico. Projeta-se que a implementação do IDP resulte em uma aceleração de até 30% no *lead time* para a entrega de novos projetos e funcionalidades, eliminando as semanas de espera tradicionais associadas à abertura de chamados (tickets) para operações.
- **Catálogo Unificado e Economia Financeira:** O módulo de catálogo do Backstage fornecerá uma visibilidade arquitetural centralizada do que está sendo construído em toda a *enterprise*. Isso extirpa a redundância de esforços, evitando que times distintos desenvolvam o mesmo software e compitam internamente. A mitigação desse retrabalho traduz-se diretamente em economia expressiva de orçamento de TI e maximização do valor da força de trabalho.
- **Governança Financeira (FinOps):** Através do tagueamento automatizado da infraestrutura, a empresa terá uma noção exata de seus gastos (*cloud spend* e *workforce*), permitindo ajustes proativos e a eliminação de recursos ociosos.
- **Maximização da Segurança com Imagens *Distroless*:** A padronização das esteiras de CI/CD garantirá aplicações *secure by default*. Espera-se uma redução drástica na superfície de ataque e otimização do tráfego de rede para rápida escalabilidade.

REFERÊNCIAS

BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol: O'Reilly Media, 2016.

CHAINGUARD. **Distroless Container Images: Securing the Software Supply Chain**. Disponível em: https://www.chainguard.dev/unchained/what-are-distroless-images. Acesso em: 9 mar. 2026.

CNCF - Cloud Native Computing Foundation. **Cloud Native Landscape**. Disponível em: https://landscape.cncf.io/. Acesso em: 9 mar. 2026.

CROSSPLANE. **The Cloud Native Control Plane**. Documentação oficial. Disponível em: https://crossplane.io/docs/. Acesso em: 9 mar. 2026.

KYVERNO. **Kubernetes Native Policy Management**. Documentação oficial. Disponível em: https://kyverno.io/docs/. Acesso em: 9 mar. 2026.

MICROSOFT. **Arquitetura do Azure Kubernetes Service (AKS)**. Microsoft Learn. Disponível em: https://learn.microsoft.com/pt-br/azure/aks/concepts-clusters-workloads. Acesso em: 9 mar. 2026.

MICROSOFT. **Integração do Azure Key Vault com o AKS via Secrets Store CSI Driver**. Microsoft Learn. Disponível em: https://learn.microsoft.com/pt-br/azure/aks/csi-secrets-store-driver. Acesso em: 9 mar. 2026.

SKELTON, M.; PAIS, M. **Team Topologies: Organizing Business and Technology Teams for Fast Flow**. Portland: IT Revolution Press, 2019.

SPOTIFY. **Backstage: An open platform for building developer portals**. Disponível em: https://backstage.io/docs/overview/what-is-backstage. Acesso em: 9 mar. 2026.